

# ComicCraft – Complete Project Code

All project coding files in one PDF. Each file is separated by its path so you can copy and paste it directly into VS Code.

## **.env.example**

```
# Required for real Gemini generation
GEMINI_API_KEY=your_google_ai_studio_api_key

# Current Gemini model defaults. You can change these without editing Python.
GEMINI_FLASH_MODEL=gemini-2.5-flash
GEMINI_PRO_MODEL=gemini-2.5-pro

# Optional Hugging Face token. Some Hub models may require authentication.
HF_TOKEN=

# Local Diffusers image generation.
# Use "placeholder" for fast UI/API testing without downloading a diffusion model.
IMAGE_BACKEND=diffusers
IMAGE_MODEL_ID=stable-diffusion-v1-5/stable-diffusion-v1-5
IMAGE_WIDTH=512
IMAGE_HEIGHT=512
IMAGE_STEPS=20
IMAGE_GUIDANCE_SCALE=7.5

APP_HOST=127.0.0.1
APP_PORT=8000
DEBUG=true
```

# README.md

# ComicCraft – AI Comic Story Creator

ComicCraft follows the project document's end-to-end architecture:

1. User submits story prompt, character, setting, tone and art style.
2. Gemini Flash creates a structured 5-panel outline.
3. Gemini Pro expands the outline into narration and dialogue.
4. Hugging Face Diffusers + Stable Diffusion creates one illustration per panel.
5. The layout builder joins text and images.
6. FPDF2 exports a multi-page PDF.
7. FastAPI + Jinja2 renders the comic preview and export flow.

The project document describes Gemini Flash/Pro and Stable Diffusion for this same pipeline. The implementation uses the current Google G

## ## 1. Requirements

- Windows 10/11, macOS, or Linux
- Python 3.11 recommended
- VS Code
- A Google Gemini API key for real text generation
- A GPU is strongly recommended for local Stable Diffusion; CPU generation can be very slow

## ## 2. VS Code setup – Windows

Open the project folder in VS Code.

Open **Terminal** → **New Terminal** and run:

```
```powershell
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```
```

If PowerShell blocks activation, use Command Prompt:

```
```bat
.venv\Scripts\activate.bat
```
```

Copy `.env.example` to `.env` and put your Gemini API key in:

```
```env
GEMINI_API_KEY=your_key_here
```
```

## ## 3. First test without downloading Stable Diffusion

For a quick UI/API test, set:

```
```env
IMAGE_BACKEND=placeholder
```
```

Then run:

```
```powershell
uvicorn app.main:app --reload
```
```

Open:

- `http://127.0.0.1:8000`
- `http://127.0.0.1:8000/docs`
- `http://127.0.0.1:8000/health`

This mode tests the complete web → Gemini → layout → PDF workflow while using generated placeholder images.

## ## 4. Real image generation

After the application works in placeholder mode, change:

```
```env
IMAGE_BACKEND=diffusers
```
```

The first image-generation request downloads the configured Diffusers model. The model is cached locally. A compatible NVIDIA GPU is reco

If the selected Hub model requires authentication, set `HF_TOKEN`.

The application loads the Diffusers pipeline once and reuses it for later panels.

## ## 5. Run

```
```powershell
.venv\Scripts\activate
uvicorn app.main:app --reload
```
```

## ## 6. Browser test

1. Open `http://127.0.0.1:8000`.

2. Enter a story prompt.
3. Enter a character name.
4. Select setting, tone and art style.
5. Click **\*\*Generate My Comic\*\***.
6. Review all 5 panels.
7. Click **\*\*Download PDF\*\***.
8. The PDF is stored in ``static/exports/``.

## ## 7. API test

Open ``http://127.0.0.1:8000/docs``.

For ``POST /generate-comic/json``, use:

```
```json
{
  "story_prompt": "A brave fox discovers a glowing tree and protects the forest from a mysterious storm.",
  "character_name": "Luna",
  "setting": "Enchanted forest",
  "tone": "dramatic",
  "art_style": "comic book"
},
```,
```

The response contains five panels and a PDF URL.

## ## 8. Image-only test

Open:

``http://127.0.0.1:8000/test-image?prompt=a%20friendly%20fox%20in%20an%20enchanted%20forest``

This tests the image-generation service independently.

## ## 9. Automated tests

With the virtual environment active:

```
```powershell
pytest -q
```,
```

The included tests intentionally do not call Gemini or download Stable Diffusion.

## ## 10. Important notes

- Never commit ``.env``; it contains your API key.
- Local Stable Diffusion is resource-intensive.
- The application catches image-model failures and creates a visible placeholder so a failed image model does not crash the entire PDF workflow.
- For production, move long-running generation to a background job/queue and store generated assets in object storage.

## ## Architecture

```
```text
Browser
  |
  v
FastAPI routes.py
  |
  +--> Gemini Flash --> 5-panel outline
  |
  +--> Gemini Pro   --> narration + dialogue
  |
  +--> Diffusers   --> panel illustrations
  |
  v
layout_builder.py
  |
  v
exporters.py / FPDF2
  |
  v
PDF + Jinja2 comic preview
```,
```

## ## Project structure

```
```text
ComicCraft/
  app/
  ■   ■■■ __init__.py
  ■   ■■■ main.py
  ■   ■■■ config.py
  ■   ■■■ routes.py
  ■   ■■■ schemas.py
  ■   ■■■ services/
  ■       ■■■ __init__.py
  ■       ■■■ gemini_flash.py
  ■       ■■■ gemini_pro.py
  ■       ■■■ image_generator.py
  ■       ■■■ layout_builder.py
  ■       ■■■ exporters.py
  ■       ■■■ workflow.py
  ■■■ static/
  ■   ■■■ css/styles.css
```,
```

```
■ ■■■ panels/
■ ■■■ exports/
■■■ templates/
■ ■■■ base.html
■ ■■■ index.html
■ ■■■ comic_preview.html
■ ■■■ export_success.html
■ ■■■ error.html
■ ■■■ image_test.html
■■■ tests/
■ ■■■ test_health.py
■ ■■■ test_layout.py
■■■ .env.example
■■■ .gitignore
■■■ pytest.ini
■■■ requirements.txt
■■■ README.md
^^^
```

**app/\_\_init\_\_.py**

## app/config.py

```
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

BASE_DIR = Path(__file__).resolve().parent.parent

class Settings(BaseSettings):
    gemini_api_key: str = ""
    gemini_flash_model: str = "gemini-2.5-flash"
    gemini_pro_model: str = "gemini-2.5-pro"

    hf_token: str = ""
    image_model_id: str = "stable-diffusion-v1-5/stable-diffusion-v1-5"
    image_backend: str = "diffusers" # diffusers | placeholder
    image_width: int = 512
    image_height: int = 512
    image_steps: int = 20
    image_guidance_scale: float = 7.5

    app_host: str = "127.0.0.1"
    app_port: int = 8000
    debug: bool = True
    max_prompt_length: int = 2000
    static_dir: str = str(BASE_DIR / "static")
    templates_dir: str = str(BASE_DIR / "templates")

    model_config = SettingsConfigDict(
        env_file=str(BASE_DIR / ".env"),
        env_file_encoding="utf-8",
        extra="ignore",
    )

settings = Settings()
```

## app/main.py

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

from app.config import settings
from app.routes import router

app = FastAPI(
    title="ComicCraft - AI Comic Story Creator",
    description="Generate 5-panel AI comics with Gemini and Stable Diffusion.",
    version="1.0.0",
)

app.mount("/static", StaticFiles(directory=settings.static_dir), name="static")
app.include_router(router)

@app.get("/health", tags=["system"])
def health():
    return {"status": "ok", "app": "ComicCraft", "version": app.version}
```

## app/routes.py

```
from pathlib import Path
from urllib.parse import quote

from fastapi import APIRouter, Form, HTTPException, Request
from fastapi.responses import FileResponse, HTMLResponse
from fastapi.templating import Jinja2Templates

from app.config import settings
from app.schemas import ComicResponse, PromptRequest
from app.services.image_generator import generate_image
from app.services.workflow import generate_comic, public_url

router = APIRouter()
templates = Jinja2Templates(directory=settings.templates_dir)

def _title(character: str) -> str:
    return f"{character}'s Comic Adventure"

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        "index.html",
        {"request": request},
    )

@router.post("/generate", response_class=HTMLResponse)
def generate(
    request: Request,
    story_prompt: str = Form(...),
    character_name: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    art_style: str = Form(...),
):
    if len(story_prompt) > settings.max_prompt_length:
        raise HTTPException(400, "Story prompt is too long.")

    try:
        layout, pdf_path = generate_comic(
            story_prompt.strip(),
            character_name.strip(),
            setting.strip(),
            tone.strip(),
            art_style.strip(),
        )
    except Exception as exc:
        return templates.TemplateResponse(
            "error.html",
            {"request": request, "error": str(exc)},
            status_code=500,
        )

    for panel in layout:
        panel["image_url"] = public_url(panel["image_path"])

    return templates.TemplateResponse(
        "comic_preview.html",
        {
            "request": request,
            "layout": layout,
            "pdf_url": public_url(pdf_path),
            "pdf_filename": Path(pdf_path).name,
            "comic_title": _title(character_name.strip()),
        },
    )

@router.post("/generate-comic/json", response_model=ComicResponse)
def generate_json(payload: PromptRequest):
    try:
        layout, pdf_path = generate_comic(
            payload.story_prompt,
            payload.character_name,
            payload.setting,
            payload.tone,
            payload.art_style,
        )
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc)) from exc

    panels = [
        {
            "panel_number": p["panel_number"],
            "title": p["title"],
            "scene_description": p["scene_description"],
            "image_prompt": p["image_prompt"],
            "caption": p["caption"],
            "narration": p["narration"],
            "image_url": public_url(p["image_path"]),
        }
        for p in layout
    ]
    return ComicResponse(
```

```

        title=_title(payload.character_name),
        panels=panels,
        pdf_url=public_url(pdf_path),
    )

@router.get("/test-image", response_class=HTMLResponse)
def test_image(request: Request, prompt: str = "a friendly fox in an enchanted forest, comic book style"):
    try:
        image_path = generate_image(prompt, 0)
        image_url = public_url(image_path)
        return templates.TemplateResponse(
            "image_test.html",
            {"request": request, "image_url": image_url, "prompt": prompt},
        )
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc)) from exc

@router.get("/download/{filename}")
def download_pdf(filename: str):
    safe_name = Path(filename).name
    pdf_path = Path(settings.static_dir) / "exports" / safe_name
    if not pdf_path.exists() or pdf_path.suffix.lower() != ".pdf":
        raise HTTPException(status_code=404, detail="PDF not found.")
    return FileResponse(
        path=pdf_path,
        media_type="application/pdf",
        filename=safe_name,
    )

@router.get("/export-success", response_class=HTMLResponse)
def export_success(request: Request, filename: str = ""):
    safe_name = Path(filename).name if filename else ""
    exists = bool(safe_name) and (Path(settings.static_dir) / "exports" / safe_name).exists()
    return templates.TemplateResponse(
        "export_success.html",
        {"request": request, "filename": safe_name, "exists": exists},
    )

```

## app/schemas.py

```
from pydantic import BaseModel, Field

class PromptRequest(BaseModel):
    story_prompt: str = Field(min_length=3, max_length=2000)
    character_name: str = Field(min_length=1, max_length=80)
    setting: str = Field(min_length=1, max_length=120)
    tone: str = Field(min_length=1, max_length=50)
    art_style: str = Field(min_length=1, max_length=80)

class PanelOutline(BaseModel):
    panel_number: int = Field(ge=1, le=5)
    title: str
    scene_description: str
    image_prompt: str

class OutlineResponse(BaseModel):
    panels: list[PanelOutline] = Field(min_length=5, max_length=5)

class StoryPanel(BaseModel):
    panel_number: int = Field(ge=1, le=5)
    caption: str
    narration: str

class StoryResponse(BaseModel):
    panels: list[StoryPanel] = Field(min_length=5, max_length=5)

class ComicPanel(BaseModel):
    panel_number: int
    title: str
    scene_description: str
    image_prompt: str
    caption: str
    narration: str
    image_url: str

class ComicResponse(BaseModel):
    title: str
    panels: list[ComicPanel]
    pdf_url: str
```

app/services/\_\_init\_\_.py

## app/services/exporters.py

```
from __future__ import annotations

import re
from datetime import datetime
from pathlib import Path

from fpdf import FPDF

from app.config import settings


def _ascii(text: str) -> str:
    # Built-in FPDF fonts are not Unicode-complete; keep PDF export reliable.
    return text.encode("latin-1", "replace").decode("latin-1")


def save_pdf(layout: list[dict], title: str = "ComicCraft Comic") -> str:
    export_dir = Path(settings.static_dir) / "exports"
    export_dir.mkdir(parents=True, exist_ok=True)

    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"comic_{stamp}.pdf"
    output = export_dir / filename

    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    for panel in layout:
        pdf.add_page()
        pdf.set_font("Helvetica", "B", 18)
        pdf.multi_cell(0, 10, _ascii(f"Panel {panel['panel_number']}: {panel['title']}"))

        image_path = Path(panel["image_path"])
        if image_path.exists():
            pdf.image(str(image_path), x=15, y=38, w=180, h=110)

        pdf.set_y(155)
        pdf.set_font("Helvetica", "I", 10)
        pdf.multi_cell(0, 6, _ascii(panel["scene_description"]))

        pdf.ln(3)
        pdf.set_font("Helvetica", "B", 11)
        pdf.multi_cell(0, 6, _ascii("Caption"))
        pdf.set_font("Helvetica", "", 10)
        pdf.multi_cell(0, 6, _ascii(panel["caption"]))

        pdf.ln(2)
        pdf.set_font("Helvetica", "B", 11)
        pdf.multi_cell(0, 6, _ascii("Narration"))
        pdf.set_font("Helvetica", "", 10)
        pdf.multi_cell(0, 6, _ascii(panel["narration"]))

    pdf.output(str(output))
    return str(output)
```

## app/services/gemini\_flash.py

```
from app.config import settings
from app.schemas import OutlineResponse

def _client():
    try:
        from google import genai
    except ImportError as exc:
        raise RuntimeError(
            "The Google GenAI SDK is not installed. Run: pip install -r requirements.txt"
        ) from exc
    if not settings.gemini_api_key:
        raise RuntimeError("GEMINI_API_KEY is missing. Add it to .env.")
    return genai.Client(api_key=settings.gemini_api_key)

def generate_outline(
    story_prompt: str,
    character_name: str,
    setting: str,
    tone: str,
    art_style: str,
) -> list[dict]:
    from google.genai import types

    prompt = f"""
You are the outline director for ComicCraft.
Create exactly 5 connected comic panels.

User story idea: {story_prompt}
Main character: {character_name}
Setting: {setting}
Tone: {tone}
Art style: {art_style}

Requirements:
- Keep the same main character throughout.
- Make the five panels form one coherent beginning, middle, and ending.
- Each panel needs a short title, scene description, and a detailed image prompt.
- The image prompt must describe composition, character appearance, environment,
  lighting, emotion, camera framing, and the requested art style.
- Do not put dialogue in image prompts; dialogue will be added separately.
- Return only the requested structured data.
""".strip()

    response = _client().models.generate_content(
        model=settings.gemini_flash_model,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=0.9,
            max_output_tokens=5000,
            response_mime_type="application/json",
            response_schema=OutlineResponse,
        ),
    )
    parsed = response.parsed or OutlineResponse.model_validate_json(response.text)
    return [panel.model_dump() for panel in parsed.panels]
```

## app/services/gemini\_pro.py

```
from app.config import settings
from app.schemas import StoryResponse

def _client():
    try:
        from google import genai
    except ImportError as exc:
        raise RuntimeError(
            "The Google GenAI SDK is not installed. Run: pip install -r requirements.txt"
        ) from exc
    if not settings.gemini_api_key:
        raise RuntimeError("GEMINI_API_KEY is missing. Add it to .env.")
    return genai.Client(api_key=settings.gemini_api_key)

def generate_story(
    outline: list[dict],
    character_name: str,
    tone: str,
) -> list[dict]:
    from google.genai import types

    outline_text = "\n".join(
        f"Panel {p['panel_number']} - {p['title']}: {p['scene_description']}"
        for p in outline
    )
    prompt = f"""
You are the senior comic writer for ComicCraft.
Expand the supplied 5-panel outline into concise, engaging comic narration.

Main character: {character_name}
Tone: {tone}

Outline:
{outline_text}

Requirements:
- Return exactly 5 story panels, matching panel numbers.
- Each panel has a short ambient caption and narration.
- Narration may include natural character dialogue using quotation marks.
- Keep continuity across panels.
- Keep each panel's text suitable for a comic page, roughly 60-100 words total.
- Do not add extra panels.
"""

    response = _client().models.generate_content(
        model=settings.gemini_pro_model,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=0.85,
            max_output_tokens=6000,
            response_mime_type="application/json",
            response_schema=StoryResponse,
        ),
    )
    parsed = response.parsed or StoryResponse.model_validate_json(response.text)
    return [panel.model_dump() for panel in parsed.panels]
```

## app/services/image\_generator.py

```
from __future__ import annotations

import re
from pathlib import Path
from threading import Lock

from PIL import Image, ImageDraw, ImageFont

from app.config import settings

_pipeline = None
_pipeline_lock = Lock()

def _safe_filename(value: str, fallback: str = "panel") -> str:
    cleaned = re.sub(r"[^a-zA-Z0-9_-]+", "_", value).strip("_")
    return (cleaned[:80] or fallback)

def _placeholder(prompt: str, output_path: Path, panel_number: int) -> str:
    image = Image.new("RGB", (settings.image_width, settings.image_height), "white")
    draw = ImageDraw.Draw(image)
    title = f"ComicCraft - Panel {panel_number}"
    draw.rectangle((12, 12, settings.image_width - 12, settings.image_height - 12), outline="black", width=4)
    draw.text((30, 35), title, fill="black")
    short = prompt[:220].replace("\n", " ")
    draw.text((30, 100), "Image backend: placeholder", fill="black")
    draw.text((30, 145), short, fill="black")
    image.save(output_path)
    return str(output_path)

def _get_pipeline():
    global _pipeline
    if _pipeline is not None:
        return _pipeline

    with _pipeline_lock:
        if _pipeline is None:
            import torch
            from diffusers import StableDiffusionPipeline

            dtype = torch.float16 if torch.cuda.is_available() else torch.float32
            kwargs = {"torch_dtype": dtype}
            if settings.hf_token:
                kwargs["token"] = settings.hf_token

            _pipeline = StableDiffusionPipeline.from_pretrained(
                settings.image_model_id,
                **kwargs,
            )
            if torch.cuda.is_available():
                _pipeline = _pipeline.to("cuda")
            else:
                _pipeline = _pipeline.to("cpu")
            _pipeline.enable_attention_slicing()
        return _pipeline

def generate_image(prompt: str, panel_number: int) -> str:
    panels_dir = Path(settings.static_dir) / "panels"
    panels_dir.mkdir(parents=True, exist_ok=True)
    filename = f"panel_{panel_number}_{_safe_filename(prompt)}.png"
    output_path = panels_dir / filename

    if settings.image_backend.lower() == "placeholder":
        return _placeholder(prompt, output_path, panel_number)

    try:
        pipe = _get_pipeline()
        image = pipe(
            prompt=prompt,
            negative_prompt="blurry, distorted face, extra limbs, text, watermark, low quality",
            width=settings.image_width,
            height=settings.image_height,
            num_inference_steps=settings.image_steps,
            guidance_scale=settings.image_guidance_scale,
        ).images[0]
        image.save(output_path)
        return str(output_path)
    except Exception as exc:
        # Keep the web app usable if the local diffusion model cannot load.
        # The error is visible in the generated placeholder rather than being hidden.
        return _placeholder(f"Image generation failed: {exc}", output_path, panel_number)
```

## app/services/layout\_builder.py

```
from pathlib import Path

def build_comic_layout(outline: list[dict], story: list[dict], image_paths: list[str]) -> list[dict]:
    story_by_number = {p["panel_number"]: p for p in story}
    layout = []

    for outline_panel, image_path in zip(outline, image_paths):
        number = outline_panel["panel_number"]
        story_panel = story_by_number[number]
        layout.append({
            "panel_number": number,
            "title": outline_panel["title"],
            "scene_description": outline_panel["scene_description"],
            "image_prompt": outline_panel["image_prompt"],
            "caption": story_panel["caption"],
            "narration": story_panel["narration"],
            "image_path": image_path,
            "image_url": "/" + str(Path(image_path).as_posix()).lstrip("/"),
        })
    return layout
```

## app/services/workflow.py

```
from pathlib import Path

from app.services.gemini_flash import generate_outline
from app.services.gemini_pro import generate_story
from app.services.image_generator import generate_image
from app.services.layout_builder import build_comic_layout
from app.services.exporters import save_pdf

def generate_comic(
    story_prompt: str,
    character_name: str,
    setting: str,
    tone: str,
    art_style: str,
) -> tuple[list[dict], str]:
    outline = generate_outline(
        story_prompt, character_name, setting, tone, art_style
    )
    story = generate_story(outline, character_name, tone)

    image_paths = []
    for panel in outline:
        image_prompt = (
            f"{panel['image_prompt']}. Comic panel, {art_style}, "
            "consistent character design, clean composition, no written text."
        )
        image_paths.append(generate_image(image_prompt, panel["panel_number"]))

    layout = build_comic_layout(outline, story, image_paths)
    pdf_path = save_pdf(layout, title=f"{character_name}'s Comic")
    return layout, pdf_path

def public_url(path: str) -> str:
    p = Path(path).as_posix()
    marker = "/static/"
    if marker in p:
        return p[p.index(marker):]
    return "/" + p.lstrip("/")
```

## requirements.txt

```
fastapi>=0.115,<1
uvicorn[standard]>=0.30,<1
jinja2>=3.1,<4
python-multipart>=0.0.9,<1
pydantic>=2.8,<3
pydantic-settings>=2.4,<3
google-generativeai>=1.60,<2
diffusers>=0.40,<1
transformers>=4.45,<5
accelerate>=1.0,<2
torch>=2.2,<3
Pillow>=10,<12
fpdf2>=2.8,<3
pytest>=8,<9
httpx>=0.27,<1
```

## static/css/styles.css

```
:root {
  --ink: #171717;
  --muted: #6b6b6b;
  --paper: #fffd8;
  --card: #ffffff;
  --line: #e8e1d5;
  --accent: #6d3df5;
  --accent-dark: #4f27bd;
  --soft: #f3efff;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  background: linear-gradient(135deg, #f8f3ff, #fffd8);
  color: var(--ink);
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
}
.site-header {
  max-width: 1120px; margin: auto; padding: 22px 24px;
  display: flex; align-items: baseline; gap: 14px;
}
.brand { font-size: 1.35rem; font-weight: 900; text-decoration: none; color: var(--ink); }
.tagline { color: var(--muted); font-size: .9rem; }
.container { max-width: 1000px; margin: 0 auto; padding: 24px; }
.hero { padding: 54px 0 28px; }
.eyebrow { font-size: .75rem; font-weight: 900; letter-spacing: .16em; color: var(--accent); }
h1 { font-size: clamp(2.1rem, 5vw, 4rem); line-height: 1.02; margin: 8px 0 16px; }
h2 { margin: 0; }
.lead { max-width: 700px; font-size: 1.1rem; color: var(--muted); line-height: 1.7; }
.card {
  background: rgba(255,255,255,.92); border: 1px solid var(--line);
  border-radius: 22px; padding: 28px; box-shadow: 0 18px 55px rgba(42, 31, 70, .08);
}
.form-card { display: grid; gap: 22px; }
label { display: grid; gap: 8px; font-weight: 750; }
input, textarea, select {
  width: 100%; border: 1px solid #d8d1c5; border-radius: 12px;
  padding: 13px 14px; background: white; font: inherit;
}
textarea { resize: vertical; }
.grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 18px; }
button, .button-link {
  border: 0; cursor: pointer; text-decoration: none; display: inline-flex;
  align-items: center; justify-content: center; gap: 8px;
}
.primary {
  background: var(--accent); color: white; padding: 13px 20px; border-radius: 12px;
  font-weight: 850; box-shadow: 0 10px 24px rgba(109,61,245,.24);
}
.primary:hover { background: var(--accent-dark); }
.secondary {
  color: var(--ink); border: 1px solid var(--line); background: white;
  padding: 12px 18px; border-radius: 12px; text-decoration: none; font-weight: 750;
}
.hint, footer, .text-link { color: var(--muted); font-size: .9rem; }
.preview-head { display: flex; justify-content: space-between; align-items: end; gap: 18px; margin: 20px 0 28px; }
.actions { display: flex; gap: 10px; flex-wrap: wrap; }
.panel { margin: 0 0 28px; }
.panel-title { display: flex; align-items: baseline; gap: 12px; margin-bottom: 18px; }
.panel-title span { color: var(--accent); font-weight: 900; }
.comic-image { width: 100%; max-height: 620px; object-fit: contain; border-radius: 14px; background: #f4f1eb; }
.scene { color: var(--muted); line-height: 1.6; }
.story-copy { border-left: 4px solid var(--accent); padding-left: 16px; }
.story-copy h3 { margin: 14px 0 4px; font-size: .9rem; text-transform: uppercase; letter-spacing: .08em; }
.story-copy p { line-height: 1.7; margin: 0; }
details { margin-top: 18px; color: var(--muted); }
details p { line-height: 1.5; }
.download-box, .success, .error { text-align: center; margin: 40px 0; }
.download-box { display: grid; gap: 14px; justify-items: center; padding: 24px; }
.success-icon { margin: 0 auto 12px; width: 60px; height: 60px; border-radius: 50%; display: grid; place-items: center; background: #e6f7 }
.text-link { display: block; margin-top: 18px; text-decoration: underline; }
footer { max-width: 1000px; margin: 40px auto; padding: 24px; text-align: center; }
@media (max-width: 720px) {
  .grid-2, .preview-head { grid-template-columns: 1fr; display: grid; }
  .preview-head { align-items: start; }
  .container { padding: 16px; }
}
```

## templates/base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}ComicCraft{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for('static', path='css/styles.css') }}">
</head>
<body>
<header class="site-header">
  <a class="brand" href="/">ComicCraft</a>
  <span class="tagline">AI Comic Story Creator</span>
</header>
<main class="container">
  {% block content %}{% endblock %}
</main>
<footer>ComicCraft · FastAPI · Gemini · Diffusers</footer>
</body>
</html>
```

## templates/comic\_preview.html

```
{% extends "base.html" %}
{% block title %}{{ comic_title }} · ComicCraft{% endblock %}
{% block content %}
<div class="preview-head">
  <div>
    <p class="eyebrow">COMIC PREVIEW</p>
    <h1>{{ comic_title }}</h1>
  </div>
  <div class="actions">
    <a class="secondary" href="/">Create another</a>
    <a class="primary button-link" href="/download/{{ pdf_filename }}">Download PDF</a>
  </div>
</div>

{% for panel in layout %}
<article class="panel card">
  <div class="panel-title">
    <span>Panel {{ panel.panel_number }}</span>
    <h2>{{ panel.title }}</h2>
  </div>
  
  <p class="scene"><em>{{ panel.scene_description }}</em></p>
  <div class="story-copy">
    <h3>Caption</h3>
    <p>{{ panel.caption }}</p>
    <h3>Narration</h3>
    <p>{{ panel.narration }}</p>
  </div>
  <details>
    <summary>Image prompt reference</summary>
    <p>{{ panel.image_prompt }}</p>
  </details>
</article>
{% endfor %}

<div class="download-box">
  <h2>Your comic is ready.</h2>
  <a class="primary button-link" href="/download/{{ pdf_filename }}">Download Your Comic as PDF</a>
  <a class="text-link" href="/export-success?filename={{ pdf_filename }}">Open export success page</a>
</div>
{% endblock %}
```

## templates/error.html

```
{% extends "base.html" %}
{% block title %}Generation Error · ComicCraft{% endblock %}
{% block content %}
<section class="card error">
  <p class="eyebrow">GENERATION ERROR</p>
  <h1>Comic generation could not finish.</h1>
  <p>{{ error }}</p>
  <a class="primary button-link" href="/">Back to ComicCraft</a>
</section>
{% endblock %}
```

## templates/export\_success.html

```
{% extends "base.html" %}
{% block title %}Export Complete · ComicCraft{% endblock %}
{% block content %}
<section class="card success">
  <div class="success-icon">✓</div>
  <p class="eyebrow">EXPORT COMPLETE</p>
  <h1>Your comic PDF is ready.</h1>
  {% if exists %}
    <p>You can download the generated PDF again or create another comic.</p>
    <a class="primary button-link" href="/download/{{ filename }}">Download PDF</a>
  {% else %}
    <p>The requested export could not be found.</p>
  {% endif %}
  <a class="text-link" href="/">Go Create Another Comic</a>
</section>
{% endblock %}
```

# templates/image\_test.html

```
{% extends "base.html" %}
{% block title %}Image Test · ComicCraft{% endblock %}
{% block content %}
<section class="card">
  <p class="eyebrow">DEVELOPER TEST</p>
  <h1>Image generation</h1>
  <p>{{ prompt }}</p>
  
</section>
{% endblock %}
```

# templates/index.html

```
{% extends "base.html" %}
{% block title %}Create a Comic · ComicCraft{% endblock %}
{% block content %}
<section class="hero">
  <div>
    <p class="eyebrow">CREATE YOUR STORY</p>
    <h1>Turn an idea into a 5-panel comic.</h1>
    <p class="lead">Describe your story, choose the mood and art style, and let ComicCraft build the panels, narration, images and PDF.</p>
  </div>
</section>

<form class="card form-card" action="/generate" method="post">
  <label>Story Prompt
    <textarea name="story_prompt" rows="5" maxlength="2000" required placeholder="A brave fox exploring an enchanted forest..."></textarea>
  </label>

  <div class="grid-2">
    <label>Main Character Name
      <input name="character_name" maxlength="80" required placeholder="Luna">
    </label>
    <label>Setting
      <select name="setting">
        <option>Enchanted forest</option>
        <option>School</option>
        <option>Space</option>
        <option>City</option>
        <option>Village</option>
      </select>
    </label>
  </div>

  <div class="grid-2">
    <label>Story Tone
      <select name="tone">
        <option>light-hearted</option>
        <option>dramatic</option>
        <option>poetic</option>
        <option>funny</option>
      </select>
    </label>
    <label>Art Style
      <select name="art_style">
        <option>anime</option>
        <option>pixel art</option>
        <option>comic book</option>
        <option>realistic</option>
      </select>
    </label>
  </div>

  <button class="primary" type="submit">■ Generate My Comic</button>
  <p class="hint">Generation can take several minutes when local Stable Diffusion is enabled.</p>
</form>
{% endblock %}
```

## tests/test\_health.py

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"

def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert "ComicCraft" in response.text
```

## tests/test\_layout.py

```
from app.services.layout_builder import build_comic_layout

def test_build_comic_layout():
    outline = [
        {"panel_number": 1, "title": "Start", "scene_description": "A forest.", "image_prompt": "fox"},
    ]
    story = [
        {"panel_number": 1, "caption": "Birds chirp.", "narration": "Luna walks forward."},
    ]
    result = build_comic_layout(outline, story, ["static/panels/panel_1.png"])
    assert result[0]["panel_number"] == 1
    assert result[0]["caption"] == "Birds chirp."
```